

High-Order Explicit Electro-Activation Solver with TNNP Ionic Model

cardiacFoam: highOrderElectroActivationFoamExplicit

May 6, 2026

1 Purpose

This report documents the solver `/home/pablo/cardiacFoam-pc/applications/solvers/highOrderElectroActivationFoamExplicit`. The solver is an explicit monodomain electro-activation solver. It is intended for benchmark-type simulations such as the Niederer et al. three-dimensional cardiac electrophysiology benchmark. The solver supports either standard finite-volume diffusion and cell-centred ionic integration, or high-order LRE reconstruction for the diffusion operator and for the volume integration of the ionic source.

The output quantities used by the benchmark are the activation time field, the activation time at point P8, and activation time samples along the diagonal from P1 to P8.

2 Governing Equation

The solver advances the monodomain equation for the transmembrane potential V_m ,

$$\frac{\partial V_m}{\partial t} = \frac{1}{\chi C_m} \nabla \cdot (\boldsymbol{\sigma} \nabla V_m) - I_{\text{ion}}(V_m, \mathbf{y}) + \frac{I_{\text{stim}}}{\chi C_m}, \quad (1)$$

where χ is the membrane surface-to-volume ratio, C_m is the membrane capacitance per unit area, $\boldsymbol{\sigma}$ is the conductivity tensor, I_{stim} is the applied stimulus current density, and $\mathbf{y}$ denotes the ionic state vector.

The TNNP model uses membrane voltage in millivolts internally. The solver field V_m is passed to the ionic model and converted as

$$V = 1000 V_m, \quad (2)$$

where V is the voltage used in the TNNP equations below.

3 Explicit Time Integration

Let K_i be a control volume. The explicit update used by the solver can be written as

$$V_{m,i}^{n+1} = V_{m,i}^n + \Delta t \left[\frac{1}{\chi C_m} L_i(V_m^n) - \bar{I}_{\text{ion},i}^n + \frac{I_{\text{stim},i}^n}{\chi C_m} \right], \quad (3)$$

where $L_i(V_m^n)$ is either the standard finite-volume diffusion operator or the high-order LRE diffusion operator. The ionic state equations are advanced after the voltage update through the selected OpenFOAM ODE solver. For the Niederer setup used here, the selected ODE method is RKF45.

4 High-Order Spatial Discretization

4.1 Diffusion Operator

When `useHighOrder_Vm` `false`, the diffusion term is evaluated with the standard finite-volume operator

$$L_i(V_m) = [\nabla \cdot (\boldsymbol{\sigma} \nabla V_m)]_i. \quad (4)$$

When `useHighOrder_Vm` `true`, LRE reconstruction is used at face quadrature points. For a face f , the high-order diffusive flux is assembled as

$$\Phi_f = |S_f| \sum_{q=1}^{N_f} w_{fq} \mathbf{n}_f \cdot (\boldsymbol{\sigma}_{\text{owner}(f)} \nabla_h V_m(\mathbf{x}_{fq})), \quad (5)$$

and the cell diffusion contribution is

$$L_i(V_m) = \frac{1}{|K_i|} \sum_{f \in \partial K_i} s_{if} \Phi_f, \quad (6)$$

where $s_{if} = +1$ for outward owner-cell flux and $s_{if} = -1$ for neighbour contributions.

4.2 Ionic Source Quadrature

The solver distinguishes the high-order reconstruction of the voltage from the high-order volume integration of the ionic source.

If `useHighOrder_Iion` `false`, the ionic model is integrated at cell centres:

$$\bar{I}_{\text{ion},i} = I_{\text{ion}}(V_{m,i}, \mathbf{y}_i). \quad (7)$$

If `useHighOrder_Iion` `true`, the ionic states are stored and evolved at the cell quadrature points $\mathbf{x}_{iq}$. The volume-averaged source is then

$$\bar{I}_{\text{ion},i} = \frac{\sum_{q=1}^{N_i} w_{iq} I_{\text{ion}}(V_{m,iq}, \mathbf{y}_{iq})}{\sum_{q=1}^{N_i} w_{iq}}. \quad (8)$$

The quadrature-point voltage is reconstructed as

$$V_{m,iq} = \begin{cases} R_{\text{LRE}}[V_m](\mathbf{x}_{iq}), & \text{useHighOrder_Vm true,} \\ V_{m,i} + \nabla V_{m,i} \cdot (\mathbf{x}_{iq} - \mathbf{x}_i), & \text{useHighOrder_Vm false.} \end{cases} \quad (9)$$

The ionic state vector at each quadrature point is advanced by

$$\frac{d\mathbf{y}_{iq}}{dt} = \mathbf{F}_{\text{TNNP}}(V_{iq}, \mathbf{y}_{iq}, t). \quad (10)$$

For writing fields such as `Ca_i`, the solver volume-averages the quadrature states back to cells:

$$\mathbf{y}_i = \frac{\sum_q w_{iq} \mathbf{y}_{iq}}{\sum_q w_{iq}}. \quad (11)$$

5 Boundary Conditions

For the standard finite-volume branch, OpenFOAM boundary fields determine the boundary contribution. For the high-order diffusion branch, `zeroGradient` and `empty` patches are treated as true no-flux boundaries:

$$\mathbf{n} \cdot \boldsymbol{\sigma} \nabla V_m = 0 \quad \text{on} \quad \Gamma_N. \quad (12)$$

In the implementation, this means that the high-order boundary face flux is set to zero on these patches:

$$\Phi_f = 0, \quad f \subset \Gamma_N. \quad (13)$$

This is important because simply reconstructing a high-order gradient at a `zeroGradient` boundary can produce a non-zero normal flux unless the Neumann condition is explicitly enforced at the high-order flux level.

Only `fixedValue` or `fixedVoltage` patches are included in the high-order voltage stencil through boundary values. `zeroGradient` patches are not included as value constraints in the high-order voltage stencil.

6 Activation Time

The activation time field is the first time at which V_m crosses the prescribed threshold V_{thr} from below. If $V_{m,i}^n < V_{\text{thr}} \leq V_{m,i}^{n+1}$, the crossing time is linearly interpolated as

$$t_{\text{act},i} = t^n + \Delta t \frac{V_{\text{thr}} - V_{m,i}^n}{V_{m,i}^{n+1} - V_{m,i}^n}. \quad (14)$$

The Niederer benchmark output point P8 is sampled from the activation-time field at

$$\mathbf{x}_{P8} = (0.02, 0.003, 0) \text{ m}. \quad (15)$$

7 TNNP Ionic Model

The solver uses the ten Tusscher–Noble–Noble–Panfilov 2004 ionic model through `ionicModel` TNNP. The state vector is

$$\mathbf{y} = [V, K_i, Na_i, Ca_i, X_{r1}, X_{r2}, X_s, m, h, j, d, f, f_{Ca}, s, r, Ca_{SR}, g]^T. \quad (16)$$

For the monodomain solver, V is imposed by the PDE and is not evolved inside the ionic ODE system unless `solveVmWithinODESolver` is enabled.

7.1 Reversal Potentials

$$E_{Na} = \frac{RT}{F} \ln \left(\frac{Na_o}{Na_i} \right), \quad (17)$$

$$E_K = \frac{RT}{F} \ln \left(\frac{K_o}{K_i} \right), \quad (18)$$

$$E_{Ks} = \frac{RT}{F} \ln \left(\frac{K_o + P_{kna} Na_o}{K_i + P_{kna} Na_i} \right), \quad (19)$$

$$E_{Ca} = \frac{1}{2} \frac{RT}{F} \ln \left(\frac{Ca_o}{Ca_i} \right). \quad (20)$$

7.2 Ionic Currents

The total ionic current used by the model is

$$I_{\text{ion}} = i_{K1} + i_{to} + i_{Kr} + i_{Ks} + i_{CaL} + i_{NaK} + i_{Na} + i_{bNa} + i_{NaCa} + i_{bCa} + i_{pK} + i_{pCa}. \quad (21)$$

The individual currents are

$$i_{Na} = g_{Na} m^3 h j (V - E_{Na}), \quad (22)$$

$$i_{bNa} = g_{bNa} (V - E_{Na}), \quad (23)$$

$$i_{K1} = g_{K1} x_{K1,\infty} \sqrt{\frac{K_o}{5.4}} (V - E_K), \quad (24)$$

$$i_{to} = g_{to} r s (V - E_K), \quad (25)$$

$$i_{Kr} = g_{Kr} \sqrt{\frac{K_o}{5.4}} X_{r1} X_{r2} (V - E_K), \quad (26)$$

$$i_{Ks} = g_{Ks} X_s^2 (V - E_{Ks}), \quad (27)$$

$$i_{bCa} = g_{bCa} (V - E_{Ca}), \quad (28)$$

$$i_{pK} = \frac{g_{pK} (V - E_K)}{1 + \exp((25 - V)/5.98)}, \quad (29)$$

$$i_{pCa} = \frac{g_{pCa} C a_i}{C a_i + K_{pCa}}. \quad (30)$$

The L-type calcium current is

$$i_{CaL} = \frac{g_{CaL} d f f_{Ca} \frac{4VF^2}{RT} (C a_i \exp(2VF/RT) - 0.341 C a_o)}{\exp(2VF/RT) - 1}. \quad (31)$$

The sodium–potassium pump and sodium–calcium exchanger are

$$i_{NaK} = \frac{P_{NaK} \frac{K_o}{K_o + K_{mk}} \frac{Na_i}{Na_i + K_{mNa}}}{1 + 0.1245 \exp(-0.1VF/RT) + 0.0353 \exp(-VF/RT)}, \quad (32)$$

$$i_{NaCa} = \frac{K_{NaCa} [\exp(\gamma VF/RT) Na_i^3 C a_o - \exp((\gamma - 1)VF/RT) Na_o^3 C a_i \alpha]}{(K_{mNa}^3 + Na_o^3)(K_{mCa} + C a_o) [1 + K_{sat} \exp((\gamma - 1)VF/RT)]}. \quad (33)$$

7.3 Gate Kinetics

Most gates use the standard form

$$\frac{dx}{dt} = \frac{x_{\infty}(V, C a_i) - x}{\tau_x(V, C a_i)}. \quad (34)$$

The implemented steady states and time constants are

$$m_\infty = [1 + \exp((-56.86 - V)/9.03)]^{-2}, \quad (35)$$

$$\alpha_m = [1 + \exp((-60 - V)/5)]^{-1}, \quad (36)$$

$$\beta_m = \frac{0.1}{1 + \exp((V + 35)/5)} + \frac{0.1}{1 + \exp((V - 50)/200)}, \quad (37)$$

$$\tau_m = \alpha_m \beta_m, \quad (38)$$

$$h_\infty = [1 + \exp((V + 71.55)/7.43)]^{-2}, \quad (39)$$

$$\alpha_h = \begin{cases} 0.057 \exp(-(V + 80)/6.8), & V < -40, \\ 0, & V \geq -40, \end{cases} \quad (40)$$

$$\beta_h = \begin{cases} 2.7 \exp(0.079V) + 310000 \exp(0.3485V), & V < -40, \\ \frac{0.77}{0.13[1 + \exp((V + 10.66)/-11.1)]}, & V \geq -40, \end{cases} \quad (41)$$

$$\tau_h = \frac{1}{\alpha_h + \beta_h}, \quad (42)$$

$$j_\infty = h_\infty, \quad (43)$$

$$\alpha_j = \begin{cases} \frac{[-25428 \exp(0.2444V) - 6.948 \cdot 10^{-6} \exp(-0.04391V)](V + 37.78)}{1 + \exp[0.311(V + 79.23)]}, & V < -40, \\ 0, & V \geq -40, \end{cases} \quad (44)$$

$$\beta_j = \begin{cases} \frac{0.02424 \exp(-0.01052V)}{1 + \exp[-0.1378(V + 40.14)]}, & V < -40, \\ \frac{0.6 \exp(0.057V)}{1 + \exp[-0.1(V + 32)]}, & V \geq -40, \end{cases} \quad (45)$$

$$\tau_j = \frac{1}{\alpha_j + \beta_j}, \quad (46)$$

$$d_\infty = [1 + \exp((-5 - V)/7.5)]^{-1}, \quad (47)$$

$$\alpha_d = \frac{1.4}{1 + \exp((-35 - V)/13)} + 0.25, \quad (48)$$

$$\gamma_{fCa} = \frac{1.4}{1 + \exp((V + 5)/5)}, \quad (49)$$

$$\gamma_d = \frac{1}{1 + \exp((50 - V)/20)}, \quad (50)$$

$$\tau_d = \alpha_d \gamma_{fCa} + \gamma_d, \quad (51)$$

$$f_\infty = [1 + \exp((V + 20)/7)]^{-1}, \quad (52)$$

$$\tau_f = 1125 \exp[-(V + 27)^2/240] + 80 + \frac{165}{1 + \exp((25 - V)/10)}, \quad (53)$$

$$X_{r1,\infty} = [1 + \exp((-26 - V)/7)]^{-1}, \quad (54)$$

$$\alpha_{xr1} = \frac{450}{1 + \exp((-45 - V)/10)}, \quad (55)$$

$$\beta_{xr1} = \frac{6}{1 + \exp((V + 30)/11.5)}, \quad (56)$$

$$\tau_{xr1} = \alpha_{xr1} \beta_{xr1}, \quad (57)$$

$$X_{r2,\infty} = [1 + \exp((V + 88)/24)]^{-1}, \quad (58)$$

$$\alpha_{xr2} = \frac{3}{1 + \exp((-60 - V)/20)}, \quad (59)$$

$$\beta_{xr2} = \frac{1.12}{1 + \exp((V - 60)/20)}, \quad (60)$$

$$\tau_{xr2} = \alpha_{xr2} \beta_{xr2}, \quad (61)$$

$$X_{s,\infty} = [1 + \exp((-5 - V)/14)]^{-1}, \quad (62)$$

The s gate depends on tissue type:

$$s_\infty = \begin{cases} [1 + \exp((V + 20)/5)]^{-1}, & \text{endocardial,} \\ 1.1 [1 + \exp((V + 28)/6)]^{-1}, & \text{M cell or epicardial.} \end{cases} \quad (68)$$

Its time constant is

$$\tau_s = \begin{cases} 85 \exp[-(V + 45)^2/320] + \frac{5}{1 + \exp((V - 20)/5)} + 3, & \text{endocardial,} \\ 1000 \exp[-(V + 67)^2/1000] + 8, & \text{M cell or epicardial.} \end{cases} \quad (69)$$

The inward rectifier auxiliary variable used by i_{K1} is

$$\alpha_{K1} = \frac{0.1}{1 + \exp[0.06((V - E_K) - 200)]}, \quad (70)$$

$$\beta_{K1} = \frac{3 \exp[0.0002((V - E_K) + 100)] + \exp[0.1((V - E_K) - 10)]}{1 + \exp[-0.5(V - E_K)]}, \quad (71)$$

$$x_{K1,\infty} = \frac{\alpha_{K1}}{\alpha_{K1} + \beta_{K1}}. \quad (72)$$

The calcium-dependent gates are

$$\alpha_{fCa} = [1 + (Ca_i/0.000325)^8]^{-1}, \quad (73)$$

$$\beta_{fCa} = 0.1 [1 + \exp((Ca_i - 0.0005)/0.0001)]^{-1}, \quad (74)$$

$$\beta_d = 0.2 [1 + \exp((Ca_i - 0.00075)/0.0008)]^{-1}, \quad (75)$$

$$f_{Ca,\infty} = \frac{\alpha_{fCa} + \beta_{fCa} + \beta_d + 0.23}{1.46}. \quad (76)$$

The f_{Ca} rate is conditionally limited:

$$\frac{df_{Ca}}{dt} = \begin{cases} 0, & f_{Ca,\infty} > f_{Ca} \text{ and } V > -60, \\ (f_{Ca,\infty} - f_{Ca})/\tau_{fCa}, & \text{otherwise.} \end{cases} \quad (77)$$

The RyR gate g follows

$$g_\infty = \begin{cases} [1 + (Ca_i/0.00035)^6]^{-1}, & Ca_i < 0.00035, \\ [1 + (Ca_i/0.00035)^{16}]^{-1}, & Ca_i \geq 0.00035, \end{cases} \quad (78)$$

$$\frac{dg}{dt} = \begin{cases} 0, & g_\infty > g \text{ and } V > -60, \\ (g_\infty - g)/\tau_g, & \text{otherwise.} \end{cases} \quad (79)$$

7.4 Concentration Equations

The sodium and potassium concentration rates are

$$\frac{dNa_i}{dt} = -\frac{(i_{Na} + i_{bNa} + 3i_{NaK} + 3i_{NaCa})C_m}{V_c F}, \quad (80)$$

$$\frac{dK_i}{dt} = -\frac{[(i_{K1} + i_{to} + i_{Kr} + i_{Ks} + i_{pK} + I_{stim}) - 2i_{NaK}]C_m}{V_c F}. \quad (81)$$

The calcium release, uptake and leak terms are

$$i_{rel} = \left[\frac{a_{rel} Ca_{SR}^2}{b_{rel}^2 + Ca_{SR}^2} + c_{rel} \right] dg, \quad (82)$$

$$i_{up} = \frac{V_{max,up}}{1 + K_{up}^2/Ca_i^2}, \quad (83)$$

$$i_{leak} = V_{leak}(Ca_{SR} - Ca_i). \quad (84)$$

The buffered calcium rates are

$$J_{Ca_i} = -\frac{(i_{CaL} + i_{bCa} + i_{pCa} - 2i_{NaCa})C_m}{2V_cF} + i_{leak} - i_{up} + i_{rel}, \quad (85)$$

$$f_{JCa_i} = \left[1 + \frac{Buf_c K_{buf,c}}{(Ca_i + K_{buf,c})^2} \right]^{-1}, \quad (86)$$

$$\frac{dCa_i}{dt} = J_{Ca_i} f_{JCa_i}, \quad (87)$$

$$J_{Ca_{SR}} = \frac{V_c}{V_{SR}}(i_{up} - i_{rel} - i_{leak}), \quad (88)$$

$$f_{JCa_{SR}} = \left[1 + \frac{Buf_{SR} K_{buf,SR}}{(Ca_{SR} + K_{buf,SR})^2} \right]^{-1}, \quad (89)$$

$$\frac{dCa_{SR}}{dt} = J_{Ca_{SR}} f_{JCa_{SR}}. \quad (90)$$

8 Numerical Protections in TNNP

The generated TNNP algebra contains logarithms, powers and divisions that are only defined for physically admissible states. The most sensitive quantities are Ca_i , Ca_{SR} , Na_i , K_i , gate variables and the removable singularity in Eq. (31) near $V = 0$.

The implemented protection is optional and controlled from `cardiacProperties`. When `TNNPNumericalProtection true`, the wrapper checks the state vector before calls to the generated TNNP algebra:

- non-finite concentrations or concentrations below `TNNPConcentrationFloor` are replaced by that floor;
- gate variables are clipped to $[0, 1]$ when `TNNP ClampGates true`;
- non-finite voltage is replaced by a resting fallback value;
- voltage is limited to $[V_{\min}, V_{\max}]$ when `TNNP ClampVmForRates true`;
- if $|V| < \varepsilon_V$, V is shifted to $\pm \varepsilon_V$ to avoid the direct evaluation of the i_{CaL} denominator at zero.

Each correction is written to `TNNP_numericalProtection.log` when `TNNPLogNumericalProtection true`. The log columns are

$$t_{ms}, \quad t_s, \quad \text{integration point}, \quad \text{phase}, \quad \text{variable}, \quad \text{old value}, \quad \text{corrected value}, \quad \text{reason}. \quad (91)$$

This makes the protection auditable: the run can be continued without losing information about which state became non-physical and where the correction was applied.

9 Niederer P8 Activation-Time Results

This section reports two-dimensional Niederer-benchmark results read from both result trees of this solver: `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderNiedererEtAl2012Ex` and `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderNiedererEtAl2012Explicit` 2/. The first folder contains the historical sweep over the requested $\Delta t \in \{10^{-2}, 5 \cdot 10^{-5}, 10^{-5}, 5 \cdot 10^{-6}\}$ s; the second folder is a more recent and more complete sweep at the single requested $\Delta t = 10^{-2}$ s. Because the explicit solver applies the CFL stability limit, the value the solver actually uses (Δt_{eff} , table at the top of the next subsection) is far smaller than the requested

one and depends on the mesh size Δx . Computational cost data come from `/usr/bin/time -v` and are reported as elapsed wall-clock time and peak resident set size (RSS). NC means that no run was found for that combination in the results tree.

The label *Vm* refers to the LRE p-level used for the high-order diffusion reconstruction, and the label *Iion* refers to the LRE p-level used for the cell-quadrature ionic-source integration. NO means that high-order is disabled for that quantity (cell-centre Laplacian / cell-centre ionic source).

9.1 P8 activation time at the requested $\Delta t = 1.0 \cdot 10^{-2}$ s

Effective time step set by the explicit CFL limit:

Δx [mm]	0.5	0.25	0.125	0.1	0.05
Δt_{eff} [s]	$1.3 \cdot 10^{-4}$	$3.3 \cdot 10^{-5}$	$8.2 \cdot 10^{-6}$	$5.2 \cdot 10^{-6}$	$1.3 \cdot 10^{-6}$

Table 1: P8 activation time [ms]: rows are LRE p-levels for $V_m \times I_{\text{ion}}$; columns are mesh sizes Δx (mm). NC = no run data found.

<i>Vm</i>	<i>Iion</i>	0.5	0.25	0.125	0.1	0.05
NO	NO	160.63	68.12	50.35	48.15	45.62
NO	p1	96.43	59.69	48.33	47.08	45.53
NO	p2	34.71	42.82	44.70	44.95	NC
NO	p3	35.33	43.46	44.67	44.92	NC
p1	NO	14.25	75.16	54.23	51.20	46.59
p1	p1	104.08	65.69	51.94	49.91	46.47
p1	p2	38.45	47.47	47.24	46.97	46.02
p1	p3	40.65	48.30	47.22	46.94	NC
p2	NO	0.00	89.11	60.13	55.60	47.85
p2	p1	159.22	74.08	56.36	53.33	47.58
p2	p2	42.88	48.89	48.50	48.14	46.56
p2	p3	48.72	50.35	48.65	48.19	NC
p3	NO	192.55	73.84	54.51	51.31	46.24
p3	p1	67.83	62.86	51.48	49.52	46.07
p3	p2	34.76	43.10	45.51	45.70	NC
p3	p3	36.02	43.80	45.54	45.69	NC

9.2 Wall-clock cost per configuration (seconds)

Total solver elapsed time as reported by `/usr/bin/time -v`.

Table 2: Solver wall-clock time [s].

<i>Vm</i>	<i>Iion</i>	0.5	0.25	0.125	0.1	0.05
NO	NO	54.6	103.9	515.0	820.9	10106.0
NO	p1	56.8	171.1	908.5	1483.4	24974.0
NO	p2	59.8	388.3	2533.2	4360.0	NC
NO	p3	79.5	522.4	3551.4	6027.1	NC
p1	NO	53.5	107.8	527.4	871.9	12183.0
p1	p1	62.0	186.7	1048.0	1645.7	22854.2
p1	p2	67.6	415.3	2691.3	4610.6	63516.6
p1	p3	91.1	555.4	3608.0	6499.0	NC
p2	NO	73.6	128.2	586.6	976.7	14360.2
p2	p1	86.5	218.2	1165.3	1834.9	23857.7
p2	p2	72.0	420.4	2838.2	4607.7	69999.9

Vm	Iion	0.5	0.25	0.125	0.1	0.05
p2	p3	103.7	575.3	3738.0	6798.1	NC
p3	NO	48.6	110.0	544.8	924.3	11838.4
p3	p1	57.8	191.0	1056.5	1846.9	24110.9
p3	p2	58.7	388.0	2819.3	4481.5	NC
p3	p3	78.1	514.5	3499.2	6341.5	NC

9.3 Peak resident set size per configuration (MiB)

Maximum physical RAM used by the solver process.

Table 3: Peak RSS [MiB].

Vm	Iion	0.5	0.25	0.125	0.1	0.05
NO	NO	148	152	165	179	297
NO	p1	149	155	182	202	388
NO	p2	152	168	234	287	NC
NO	p3	154	174	261	329	NC
p1	NO	148	154	177	198	377
p1	p1	150	158	200	234	516
p1	p2	153	170	254	318	855
p1	p3	155	176	282	361	NC
p2	NO	149	155	180	204	398
p2	p1	150	159	217	260	622
p2	p2	153	173	271	345	961
p2	p3	155	180	298	388	NC
p3	NO	150	160	202	236	524
p3	p1	151	174	277	354	995
p3	p2	155	188	331	439	NC
p3	p3	156	195	356	482	NC

10 Implementation Notes

The most relevant implementation details are:

- `useHighOrder_Vm` controls the LRE high-order diffusion reconstruction.
- `useHighOrder_Iion` controls whether TNNP states are evolved at cell quadrature points and whether I_{ion} is volume-integrated with those points.
- `useHighOrder_states` is still accepted as a legacy dictionary keyword, but the current solver interprets it as the high-order I_{ion} quadrature switch.
- `activationTime` is written as a field and can be viewed in ParaView.
- `P8_activationTime.dat` and `diagonalActivationTime.dat` are written for benchmark postprocessing.